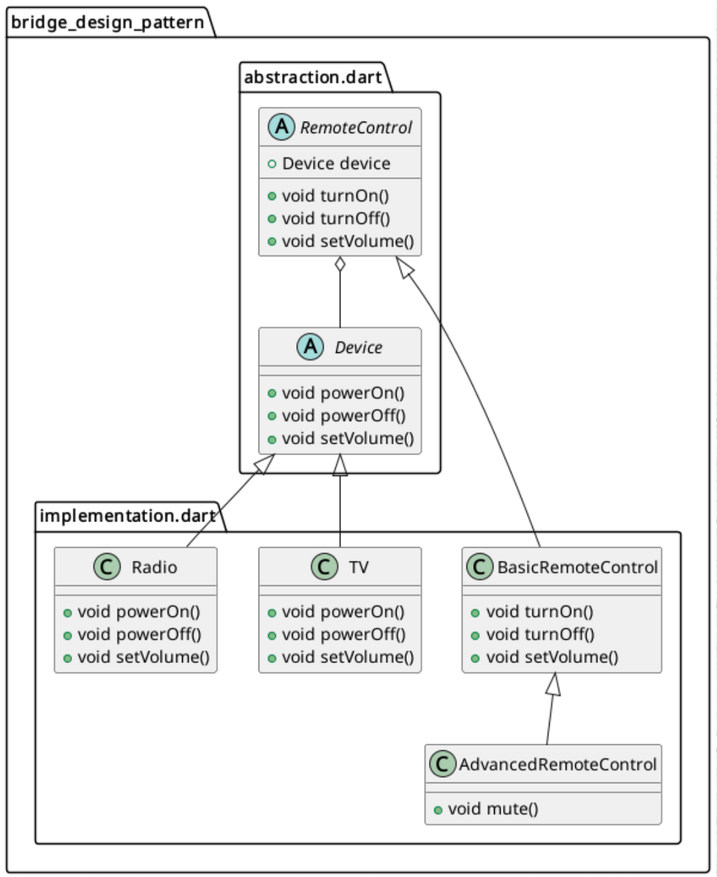
Patrón estructural

BRIDGE

Título Principal: Patrón de Diseño Estructural: BRIDGE (Puente)

PATRON BRIDGE



Bridge es un patrón de diseño estructural que te permite dividir una clase grande, o un grupo de clases estrechamente relacionadas, en dos jerarquías separadas **(abstracción e implementación)** que pueden desarrollarse independientemente la una de la otra.

Su objetivo es evitar que una clase tenga que **crear múltiples subclases** para representar todas las combinaciones posibles de características.

"No multipliques tus problemas con herencias infinitas; conéctalos inteligentemente construyendo un puente."

DOS DIMENSIONES QUE CAMBIAN

1

La abstracción

Una misma idea puede tener varias versiones visibles para el cliente.

2

La implementación

La forma de ejecutarla también puede variar.

- Ejemplo sin Bridge: Si tenemos 3 tipos de inmuebles y 2 tipos de contratos, la herencia nos obligaría a programar 6 clases rígidas (ApartamentoEstricto, CasaEstricta, PensionadoFlexible, etc.).
- Si el proyecto crece, las clases se multiplican sin control.

¿Por qué se usa en el desarrollo? Por la flexibilidad absoluta. Se usa para poder mezclar y combinar características dinámicamente sin modificar código existente.

“ Si mañana inventamos un nuevo tipo de contrato, no tenemos que tocar ni alterar las clases de los apartamentos o las casas. Ambas partes evolucionan por separado.”

PROPÓSITO / MOTIVACIÓN

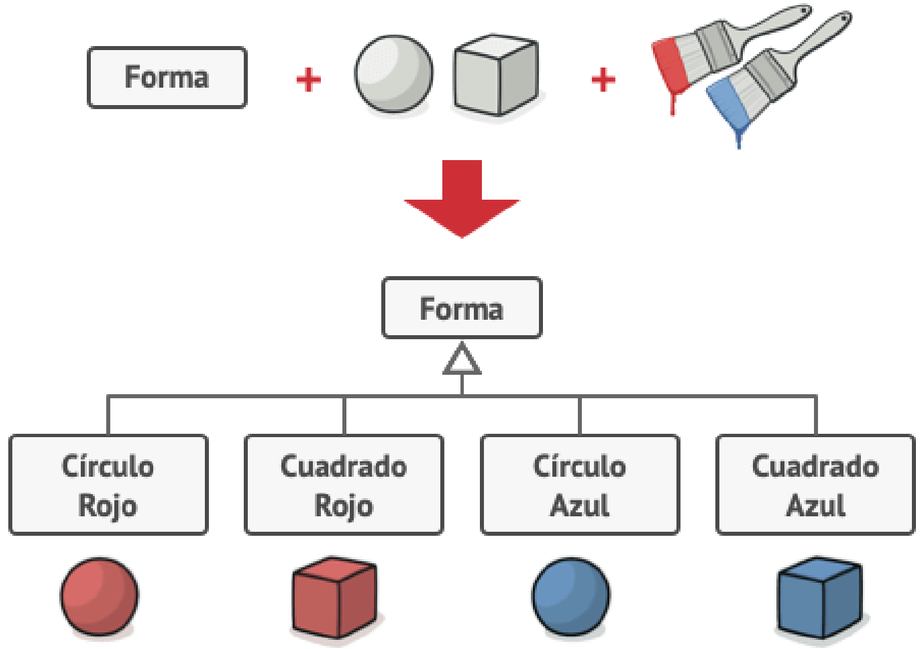
El Antipatrón detectado: "Explosión Combinatoria de Clases"

La causa: Abuso de la herencia rígida para combinar dos conceptos distintos.

El Problema en ROOMIES: Queríamos combinar la Estructura del Inmueble (Apartamento, Casa, Pensionado) con sus Reglas de Contrato (Estricta, Flexible).

La Deuda Técnica (Sin Bridge): Nos obligaba a crear una clase nueva o demasiadas por cada combinación posible:

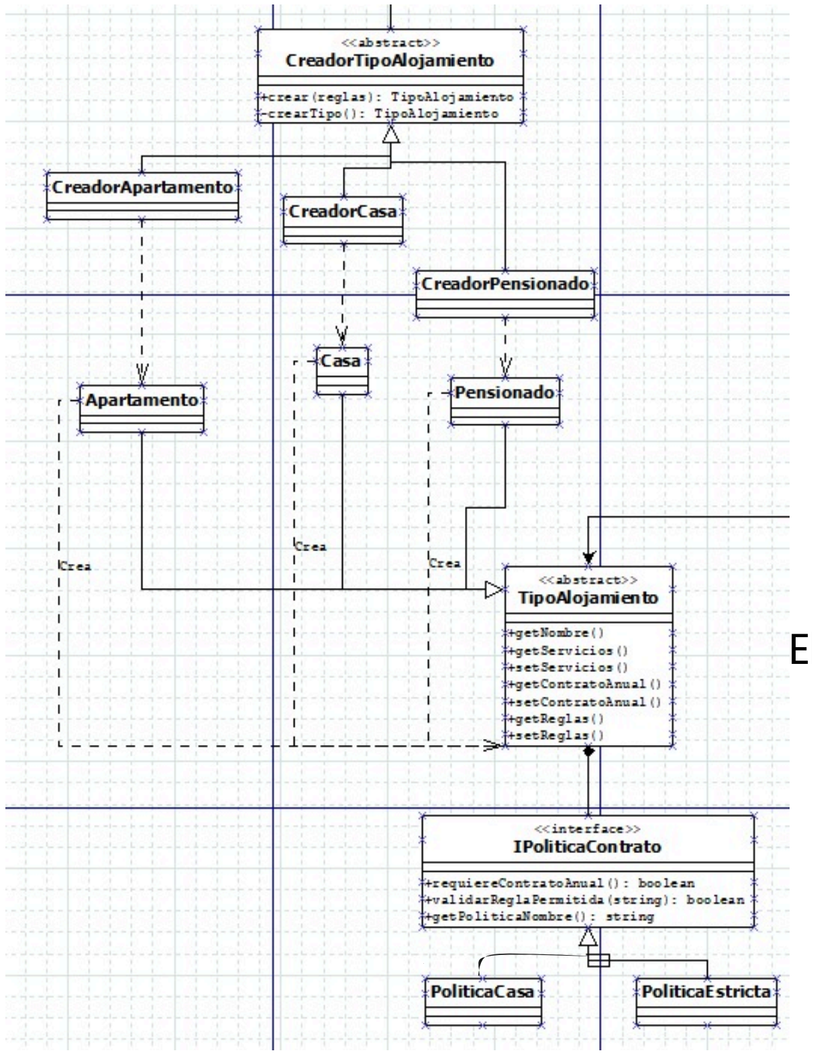
1. ApartamentoPoliticaEstricta
2. CasaPoliticaEstricta
3. ApartamentoPoliticaFlexible... (Sistema Inmantenible).

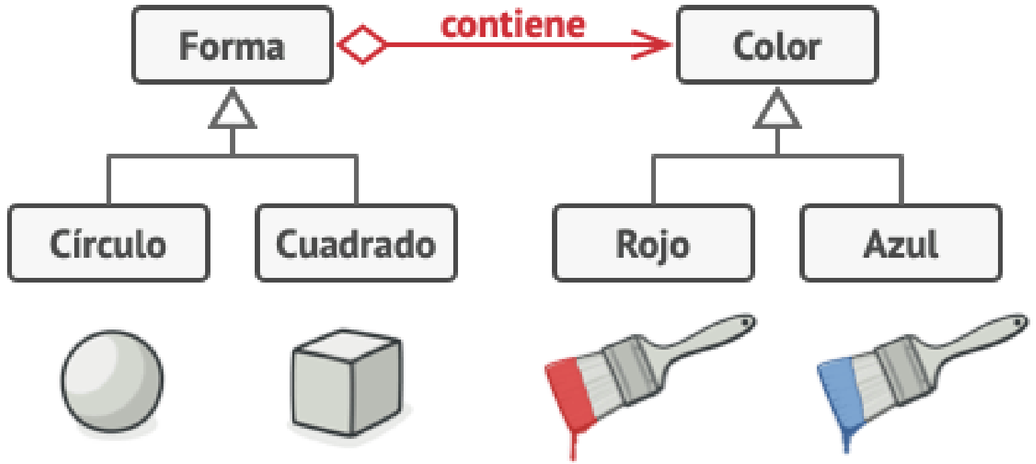


El número de combinaciones de clase crece en progresión geométrica.

El patrón Bridge resuelve este problema separando las dimensiones en dos jerarquías independientes: la Abstracción y la Implementación, conectándolas a través de una referencia (el "puente").

APLICANDO EL PATRON BRIDGE





La Decisión Arquitectónica: Decidimos implementar el patrón Bridge para limpiar el modelo,

El Resultado: Pasamos de un modelo rígido a uno dinámico, donde podemos cambiar el contrato de un inmueble en tiempo de ejecución sin alterar el código.

Detectamos que estábamos creando un volumen excesivo de clases altamente acopladas, con relaciones innecesarias y poco coherentes con nuestro modelo de negocio.

APLICABILIDAD / CONTEXTO DE APLICABILIDAD

- **Crecimiento independiente:** Tanto las abstracciones (la interfaz de alto nivel que ve el usuario) como sus implementaciones deben extenderse mediante subclases sin que esto afecte a la otra parte.

- **Evitar un enlace permanente entre abstracción e implementación:** Útil cuando la implementación puede seleccionarse o cambiarse en tiempo de ejecución.

- **Cambios transparentes para el cliente:** Cuando los cambios en la implementación de una clase no deben tener impacto en el código cliente (es decir, el cliente no necesita recompilarse).

- **Evitar la explosión de clases:** Cuando tienes una proliferación de clases debido a una combinación de múltiples dimensiones de funcionalidad.

PROS Y CONTRAS

CONTRAS

- AÑADE COMPLEJIDAD INICIAL.

Puede ser que el código se complique si aplicas el patrón a una clase muy cohesionada.

- PORQUE EL APARTAMENTO YA NO SABE QUÉ REGLAS TIENE

PRO

- PRO (ESCALABILIDAD): CRECIMIENTO INDEPENDIENTE. PODEMOS CREAR 100 CONTRATOS NUEVOS SIN TOCAR NI DAÑAR LAS CLASES DE LOS INMUEBLES (APARTAMENTO, CASA).

PREGUNTA PARA LA CLASE: BASADOS EN EL PATRÓN BRIDGE QUE ACABAMOS DE APLICAR, ¿CUÁL ES LA FORMA CORRECTA DE INTEGRAR ESTO AL CÓDIGO?

LIMITACIONES DEL PATRON

EL ESCENARIO: ROOMIES SE ESTÁ EXPANDIENDO. EL EQUIPO DE NEGOCIOS NOS PIDE AGREGAR UN NUEVO TIPO DE INMUEBLE LLAMADO GLAMPING Y UNA NUEVA REGLA LEGAL LLAMADA POLITICAMASCOTAS.



1. **A)** CREAR MÚLTIPLES CLASES COMBINADAS: GLAMPINGMASCOTAS, GLAMPINGESTRICTO Y GLAMPINGCASA.

2. **B)** MODIFICAR LA CLASE BASE TIPOALOJAMIENTO PARA AGREGARLE BLOQUES IF/ELSE QUE VALIDEN SI ES UN GLAMPING Y SI ACEPTA MASCOTAS.

3. **C)** CREAR SOLO DOS CLASES NUEVAS: GLAMPING (QUE HEREDE DE TIPOALOJAMIENTO) Y POLITICAMASCOTAS (QUE IMPLEMENTE IPOLITICACONTRATO).

4. **D)** DEJARLO ASI COMO SEA, SIN AJUSTAR NADA.

PREGUNTA PARA LA CLASE: BASADOS EN EL PATRÓN BRIDGE QUE ACABAMOS DE APLICAR, ¿CUÁL ES LA FORMA CORRECTA DE INTEGRAR ESTO AL CÓDIGO?

LIMITACIONES DEL PATRON

EL ESCENARIO: ROOMIES SE ESTÁ EXPANDIENDO. EL EQUIPO DE NEGOCIOS NOS PIDE AGREGAR UN NUEVO TIPO DE INMUEBLE LLAMADO GLAMPING Y UNA NUEVA REGLA LEGAL LLAMADA POLITICAMASCOTAS.



RESPUESTA CORRECTA: C ✅

CON EL PATRÓN BRIDGE, EL CÓDIGO FUNCIONA COMO PIEZAS DE LEGO.

❌ LA **A** CREA COMBINACIONES INFINITAS (EXPLOSIÓN DE CLASES).

❌ LA **B** DAÑA EL CÓDIGO VIEJO QUE YA FUNCIONABA.

✅ **LA C ES LA SOLUCIÓN: SOLO FABRICAMOS DOS PIEZAS NUEVA**S (GLAMPING Y POLITICAMASCOTAS) QUE ENCAJAN AUTOMÁTICAMENTE CON TODO LO QUE YA EXISTE EN ROOMIES. ¡CRECIMIENTO FÁCIL Y SIN ROMPER NADA!

Repositorio GitHub

https://github.com/Sergio5423/roomies

